

**TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**



HỌC PHẦN: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

BÁO CÁO ĐỒ ÁN

**ĐỀ TÀI: DỊCH MÁY ANH-PHÁP VỚI MÔ HÌNH
ENCODER-DECODER LSTM**

Giảng viên hướng dẫn:
PGS.TS. Nguyễn Tuấn Đăng

Nhóm sinh viên thực hiện:
3123410231 – Lê Đoàn Kim Ngân
3123410250 – Lâm Tú Nhi

TP.HCM THÁNG 12/2025

Lời nói đầu

Trong bối cảnh công nghệ thông tin và trí tuệ nhân tạo phát triển nhanh chóng, xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) và học sâu (Deep Learning) ngày càng đóng vai trò quan trọng trong việc giải quyết nhiều bài toán thực tiễn. Các ứng dụng tiêu biểu có thể kể đến như dịch máy, phân tích cảm xúc, tóm tắt văn bản hay các hệ thống trợ lý ảo thông minh. Trong số đó, dịch máy (Machine Translation – MT) là một hướng nghiên cứu nổi bật, góp phần giảm thiểu rào cản ngôn ngữ và thúc đẩy giao tiếp, học tập cũng như trao đổi thông tin trong môi trường toàn cầu hóa.

Các phương pháp dịch máy truyền thống dựa trên luật hoặc mô hình thống kê tuy đã đạt được một số kết quả nhất định, nhưng vẫn gặp nhiều khó khăn khi xử lý các câu có độ dài lớn, ngữ cảnh phức tạp hoặc khác biệt về cấu trúc ngữ pháp giữa các ngôn ngữ. Sự ra đời của các mô hình học sâu, đặc biệt là kiến trúc Sequence-to-Sequence (Seq2Seq) kết hợp với mạng LSTM (Long Short-Term Memory), đã mở ra hướng tiếp cận mới cho bài toán dịch máy. Nhờ khả năng ghi nhớ thông tin dài hạn, LSTM giúp mô hình nắm bắt tốt hơn mối quan hệ ngữ cảnh trong chuỗi, từ đó cải thiện đáng kể chất lượng bản dịch. Ngoài ra, việc tích hợp cơ chế Attention còn cho phép mô hình tập trung vào các thành phần quan trọng của câu nguồn trong quá trình sinh câu đích, giúp bản dịch trở nên chính xác và tự nhiên hơn.

Xuất phát từ những cơ sở đó, đề tài “Xây dựng mô hình LSTM dịch câu từ tiếng Anh sang tiếng Pháp” được thực hiện nhằm áp dụng các kiến thức đã học về học sâu và xử lý ngôn ngữ tự nhiên vào một bài toán thực tế. Nội dung đồ án tập trung vào việc xây dựng mô hình dịch máy dựa trên kiến trúc Encoder–Decoder, tiến hành huấn luyện mô hình trên dữ liệu song ngữ, đồng thời đánh giá chất lượng dịch thông qua chỉ số BLEU score.

Lời cảm ơn

Trong quá trình thực hiện báo cáo cuối khóa, chúng em xin bày tỏ lòng biết ơn sâu sắc tới PGS.TS. Nguyễn Tuấn Đăng đã tận tình hướng dẫn, hỗ trợ và giúp chúng em hoàn thiện đồ án. Những chỉ dẫn quý báu của thầy đã giúp nhóm tránh được nhiều sai sót và nâng cao chất lượng đồ án, hoàn thành đồ án đầy đủ và hoàn thiện hơn.

Xin cảm ơn nhà trường và các thầy cô trong khoa Công nghệ thông tin đã tạo điều kiện thuận lợi nhất để nhóm hoàn thành bài báo cáo. Đồng thời, lời cảm ơn sâu sắc cũng xin được gửi tới bạn bè, gia đình và những người thân yêu, những người đã luôn đồng hành, động viên và thấu hiểu tầm quan trọng của đồ án trong suốt quá trình học tập.

Đồ án này không chỉ là cơ hội để vận dụng những kiến thức đã được tiếp thu mà còn là dịp để chúng em rèn luyện kỹ năng triển khai thực tiễn và tư duy suy luận. Sự hỗ trợ nhiệt tình từ các thầy cô và nhân sự trong Khoa đã đóng vai trò vô cùng quan trọng, giúp chúng em vượt qua những khó khăn và phát triển ý tưởng một cách hiệu quả.

Do hạn chế về kiến thức và kinh nghiệm, bài báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những góp ý từ thầy để có thể hoàn thiện báo cáo tốt hơn. Cuối cùng, chúng em xin gửi lời cảm ơn chân thành đến thầy vì đã luôn quan tâm, hướng dẫn và tạo điều kiện để chúng em hoàn thành đồ án một cách nghiêm túc. Chúng em cũng xin bày tỏ lòng biết ơn tới các tác giả, các nguồn tài liệu và các trang web giáo dục đã mang đến những thông tin quý báu, góp phần quan trọng vào thành công của bài báo cáo.

Trân trọng.

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Mục lục

Giới thiệu chung về đề án.....	7
1. Bối cảnh dịch máy (Machine Translation – MT).....	7
2. Mục tiêu đề án.....	7
3. Ý nghĩa học thuật và thực tiễn.....	7
4. Dữ liệu.....	8
Phương pháp thực hiện.....	9
1. Pipeline toàn hệ thống.....	9
2. Sơ đồ kiến trúc mô hình.....	9
2.1. Sơ đồ kiến trúc mô hình Seq2Seq encoder–decoder không attention.....	9
2.2. Kiến trúc mạng LSTM.....	10
2.3. Mô hình Seq2Seq Encoder–Decoder với cơ chế Attention.....	12
2.4. Beam Search Decoding.....	12
3. Tiền xử lý dữ liệu.....	13
3.1. Tokenization.....	13
3.2. Xây dựng từ điển.....	14
3.3. Padding & Packing.....	14
3.4. DataLoader.....	16
4. Xây dựng mô hình.....	16
4.1. Encoder LSTM.....	16
4.2. Decoder LSTM.....	17
4.3. Attention (Luong dot).....	17
5. Huấn luyện mô hình.....	18
5.1. Giới thiệu Baseline.....	18
5.2. Huấn luyện mô hình Baseline.....	18
5.3. Quy trình huấn luyện mô hình.....	19
6. Dự đoán.....	20
6.1. Greedy Decoding.....	20
6.2. Beam Search Decoding.....	20
6.3 Dịch câu mới.....	21
7. Thảo luận.....	21
8. Kết luận.....	22
Thực nghiệm và đánh giá.....	23
1. Biểu đồ train/validation loss.....	23
1.1. Biểu đồ mô hình cơ bản, chưa tích hợp Attention và Beam Search.....	23
1.2. Biểu đồ mô hình được tích hợp cơ chế Attention.....	24
1.3. Biểu đồ mô hình được tích hợp cơ chế Attention và Beam Search.....	25
2. BLEU Score.....	26

Kết quả dịch & Phân tích lỗi.....	28
Ví dụ 1: A man in an orange hat starring at something.....	28
❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.....	28
❖ Lần 2: Mô hình được tích hợp cơ chế Attention.....	28
❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.....	29
Ví dụ 2: A Boston Terrier is running on lush green grass in front of a white fence.....	29
❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.....	29
❖ Lần 2: Mô hình được tích hợp cơ chế Attention.....	30
❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.....	30
Ví dụ 3: A girl in karate uniform breaking a stick with a front kick.....	31
❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.....	31
❖ Lần 2: Mô hình được tích hợp cơ chế Attention.....	31
❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.....	31
Ví dụ 4: Five people wearing winter jackets and helmets stand in the snow, with snowmobiles in the background.....	32
❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.....	32
❖ Lần 2: Mô hình được tích hợp cơ chế Attention.....	32
❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.....	33
Ví dụ 5: People are fixing the roof of a house.....	33
❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.....	33
❖ Lần 2: Mô hình được tích hợp cơ chế Attention.....	34
❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.....	34
Phụ lục.....	36
Link code, data, checkpoint: NLP Sources.....	36
Mã nguồn:.....	36
1. Cài đặt môi trường và tiền xử lý dữ liệu:.....	36
1.1. Import thư viện:.....	36
1.2. Cấu hình tham số:.....	36
1.3. Tokenization:.....	37
1.4. Xây dựng từ điển:.....	37
1.5. Hàm chuyển câu thành chuỗi token ID:.....	37
1.6. Translation Dataset:.....	37
1.7. Collate fn với Padding & Packing:.....	38
1.8. DataLoader:.....	38

2. Xây dựng mô hình:.....	38
2.1. Encoder:.....	38
2.2. Attention:.....	39
2.3. Decoder:.....	39
2.4. Seq2Seq:.....	40
3. Huấn luyện mô hình:.....	40
3.1. Hàm huấn luyện một epoch:.....	40
3.2. Hàm đánh giá mô hình trên tập validation:.....	41
3.3. Hàm huấn luyện toàn bộ mô hình và in ra biểu đồ train/val loss:.....	41
4. Dự đoán:.....	43
4.1. Greedy Decoding:.....	43
4.2. Beam Search Decoding:.....	44
5. Đánh giá BLEU:.....	45
6. Dịch thử:.....	46
Tài liệu tham khảo.....	47

Giới thiệu chung về đề án

1. Bối cảnh dịch máy (Machine Translation – MT)

Dịch máy là một bài toán quan trọng trong **Xử lý ngôn ngữ tự nhiên (NLP)**, đóng vai trò then chốt trong việc giảm rào cản ngôn ngữ và hỗ trợ giao tiếp toàn cầu. Trước đây, các hệ thống dịch dựa trên luật hoặc thống kê còn nhiều hạn chế. Sự phát triển của **mô hình học sâu Seq2Seq với kiến trúc Encoder–Decoder** đã mang lại bước tiến lớn, giúp bản dịch chính xác và tự nhiên hơn.

Trong đề án này, chúng em tập trung xây dựng từ đầu một mô hình **Encoder–Decoder LSTM** để dịch từ tiếng Anh sang tiếng Pháp. Quá trình thực hiện bao gồm xử lý dữ liệu song ngữ, huấn luyện mô hình, đánh giá bằng BLEU score và phân tích lỗi dịch. Mục tiêu là hiểu rõ cơ chế hoạt động của Seq2Seq, vai trò của context vector, đồng thời rèn luyện kỹ năng triển khai mô hình học sâu bằng PyTorch.

2. Mục tiêu đề án

Trong đề án này, chúng em sẽ triển khai từ đầu một mô hình **Encoder–Decoder LSTM** để dịch từ tiếng Anh sang tiếng Pháp. Quá trình thực hiện bao gồm:

- Xử lý dữ liệu song ngữ (tokenization, xây dựng từ điển, padding/packing).
- Huấn luyện mô hình bằng PyTorch với các tham số phù hợp.
- Đánh giá chất lượng dịch bằng BLEU score.
- Phân tích lỗi dịch và đề xuất cải tiến (ví dụ: attention, beam search).

Mục tiêu chính là hiểu rõ cách hoạt động của mô hình Seq2Seq, nắm vững cơ chế của LSTM trong việc xử lý chuỗi, và rèn luyện kỹ năng triển khai mô hình học sâu từ dữ liệu đến đánh giá.

3. Ý nghĩa học thuật và thực tiễn

- **Học thuật:** Đề án giúp củng cố kiến thức về mạng nơ-ron hồi tiếp (RNN), đặc biệt là LSTM, cũng như cách áp dụng chúng trong bài toán dịch máy. Đây là nền tảng để tiếp cận các mô hình tiên tiến hơn như Attention hay Transformer.
- **Thực tiễn:** Việc xây dựng một hệ thống dịch máy cơ bản giúp hiểu rõ quy trình phát triển ứng dụng NLP, từ xử lý dữ liệu đến huấn luyện và đánh giá mô hình. Kinh nghiệm này có thể áp dụng trong nhiều lĩnh vực như dịch thuật tự động, chatbot đa ngôn ngữ, hay các công cụ hỗ trợ học ngoại ngữ.

4. Dữ liệu

Trong đồ án này, dữ liệu đầu vào chính là các cặp câu song ngữ **tiếng Anh – tiếng Pháp** được sử dụng để huấn luyện và đánh giá mô hình dịch máy.

Nguồn dữ liệu:

Dataset chính thức: Multi30K (English–French) với khoảng 29.000 cặp câu huấn luyện, 1.000 cặp câu validation và 1.000 cặp câu test. Đây là tập dữ liệu gồm các câu ngắn (10–15 từ), phù hợp để chạy trên CPU/GPU cơ bản.

Phương pháp thực hiện

1. Pipeline toàn hệ thống

Hệ thống dịch máy Anh–Pháp được triển khai thông qua một pipeline xử lý hoàn chỉnh, bắt đầu từ dữ liệu thô và kết thúc ở quá trình sinh bản dịch.

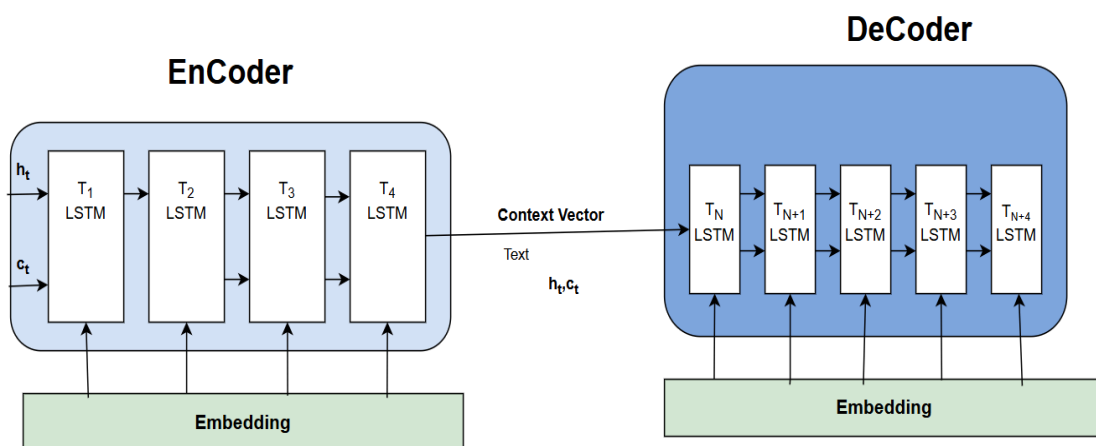
Trước tiên, dữ liệu song ngữ được đưa vào giai đoạn tiền xử lý, bao gồm tách từ (tokenization), chuẩn hóa và chuyển đổi từng token thành chỉ số số nguyên thông qua bộ từ vựng. Các câu sau đó được căn chỉnh độ dài bằng kỹ thuật padding/packing và tổ chức thành batch thông qua DataLoader.

Tiếp theo, dữ liệu đã xử lý được đưa vào mô hình Seq2Seq gồm Encoder LSTM và Decoder LSTM kết hợp cơ chế Attention. Trong giai đoạn huấn luyện, mô hình học cách ánh xạ câu nguồn sang câu đích thông qua lan truyền ngược và tối ưu tham số bằng hàm mất mát Cross-Entropy.

Sau khi huấn luyện, mô hình được đánh giá bằng BLEU Score trên bộ dữ liệu kiểm thử để đo mức độ chính xác của bản dịch. Khi suy luận trên câu mới, hệ thống áp dụng Beam Search để lựa chọn chuỗi từ tối ưu, giúp bản dịch tự nhiên và sát nghĩa hơn so với phương pháp Greedy.

2. Sơ đồ kiến trúc mô hình

2.1. Sơ đồ kiến trúc mô hình Seq2Seq encoder–decoder không attention.



Kiến trúc Encoder nhận câu nguồn, qua Embedding và LSTM để tạo hidden và cell state

Đầu tiên mà mô hình Seq2Seq encoder-decoder không dùng cơ chế Attention được dùng trong baseline. Với hai thành phần chính bao gồm:

Encoder: Chịu trách nhiệm mã hóa câu nguồn

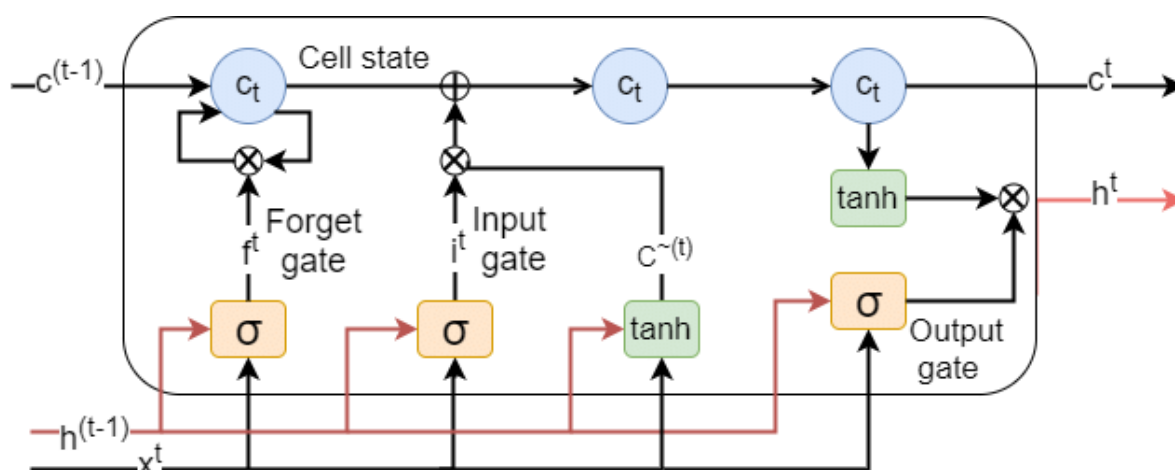
Encoder nhận vào chuỗi câu nguồn: $X = x_1, x_2, \dots, x_T$

Mỗi từ x_t được ánh xạ thành vector embedding và lần lượt đưa vào mạng LSTM Encoder theo thứ tự thời gian. Sau khi xử lý toàn bộ chuỗi đầu vào, Encoder sinh ra h_T (Trạng thái ẩn cuối cùng), c_T (Trạng thái bộ nhớ cuối cùng)

Decoder: Sinh câu đích dựa trên thông tin đã được mã hóa.

Decoder sẽ sử dụng cặp trạng thái (h_T, c_T) làm trạng thái khởi tạo ban đầu. Tại mỗi bước thời gian, Decoder sẽ nhận một từ đầu vào bắt đầu bằng ký hiệu < sos >. Sau đó dự đoán từ tiếp theo trong câu đích và sinh ra phân phối xác suất trên toàn bộ từ vựng. Cuối cùng là chọn từ có xác suất cao nhất làm đầu ra. Quá trình sẽ tiếp tục cho đến khi sinh ra ký hiệu < eos > hoặc đạt độ dài tối đa

2.2. Kiến trúc mạng LSTM



LSTM (Long Short-Term Memory) là một biến thể cải tiến của mạng nơ-ron hồi quy RNN, được thiết kế để khắc phục hiện tượng mất dần gradient khi xử lý chuỗi dài. Khác với RNN truyền thống, LSTM có khả năng ghi nhớ thông tin trong thời gian dài nhờ cấu trúc đặc biệt gồm ba cổng điều khiển.

Mỗi cell LSTM chứa hai trạng thái:

- **Trạng thái ẩn h_t** : lưu thông tin ngắn hạn
- **Trạng thái bộ nhớ c_t** : lưu thông tin dài hạn

Ba cổng điều khiển hoạt động như sau:

- **Forget Gate** – quyết định loại bỏ bao nhiêu thông tin từ trạng thái trước:

$$f_t = \sigma(U_f * x_t + W_f * h_{t-1} + b_f)$$

- **Input Gate và Cell candidate** – xác định lượng thông tin mới cần lưu vào bộ nhớ:

$$i_t = \sigma(U_i * x_t + W_i * h_{t-1} + b_i)$$

Song song với đó, LSTM sẽ tính toán Cell candidate - đại diện cho thông tin mới được đề xuất:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

- **Output Gate** – quyết định phần thông tin nào được xuất ra làm trạng thái ẩn:

$$o_t = \sigma(U_o * x_t + W_o * h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

- **Cập nhật trạng thái bộ nhớ** - Trạng thái bộ nhớ tại thời điểm ttt được cập nhật bằng cách kết hợp thông tin cũ và mới:

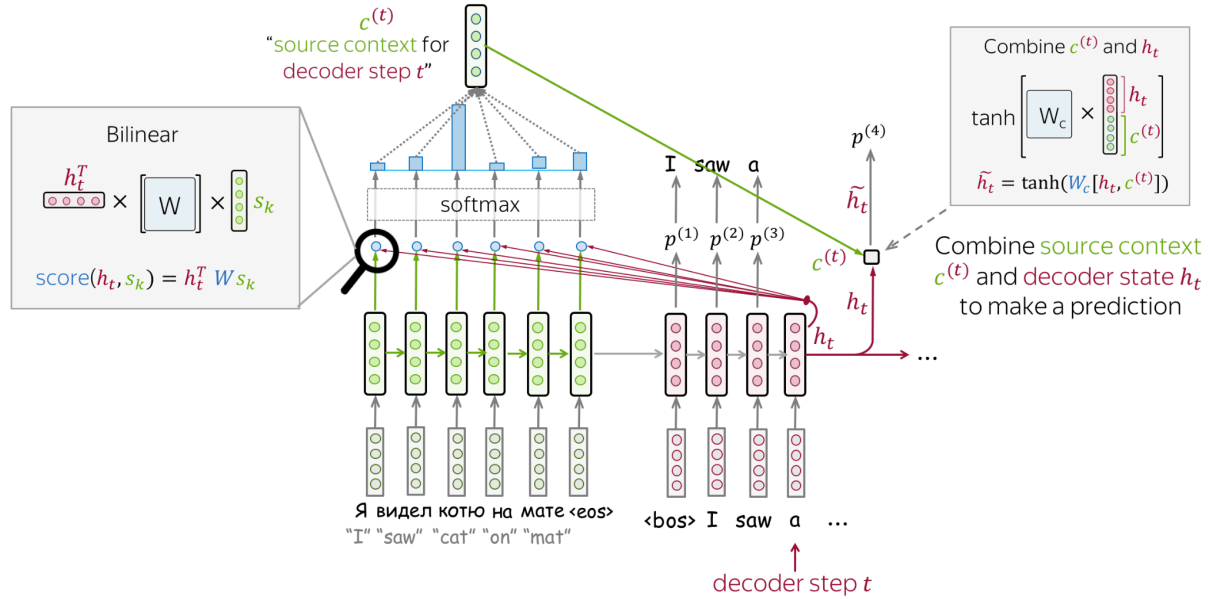
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Trạng thái ẩn được sử dụng làm đầu ra của LSTM tại thời điểm t, đồng thời được truyền sang bước thời gian tiếp theo.

Ba cổng này cho phép LSTM kiểm soát việc ghi nhớ, cập nhật và loại bỏ thông tin theo từng bước thời gian.

Trong mô hình Encoder–Decoder: Encoder LSTM trích rút đặc trưng ngữ nghĩa từ câu nguồn. Decoder LSTM sinh câu đích từng từ một dựa trên trạng thái mã hóa. Nhờ khả năng lưu giữ dài hạn, LSTM hoạt động hiệu quả hơn RNN truyền thống khi xử lý các câu dài hoặc có cấu trúc phức tạp.

2.3. Mô hình Seq2Seq Encoder–Decoder với cơ chế Attention



Để khắc phục hạn chế của mô hình Seq2Seq không Attention, cơ chế Attention được đề xuất nhằm cho phép Decoder tập trung vào những phần quan trọng của câu nguồn tại mỗi bước sinh từ.

Thay vì chỉ sử dụng trạng thái cuối cùng của Encoder, Attention cho phép Decoder truy cập toàn bộ các hidden state của Encoder:

$$H = h_1, h_2, \dots, h_T$$

Thì tại mỗi bước thời gian t , Decoder sẽ so sánh trạng thái ẩn hiện tại với hidden state của Encoder. Sau đó tính trọng số Attention. Cuối cùng là tổng hợp thông tin thành vector ngữ cảnh động

Attention Luong (Dot-product)

Trọng số Attention được tính như sau:

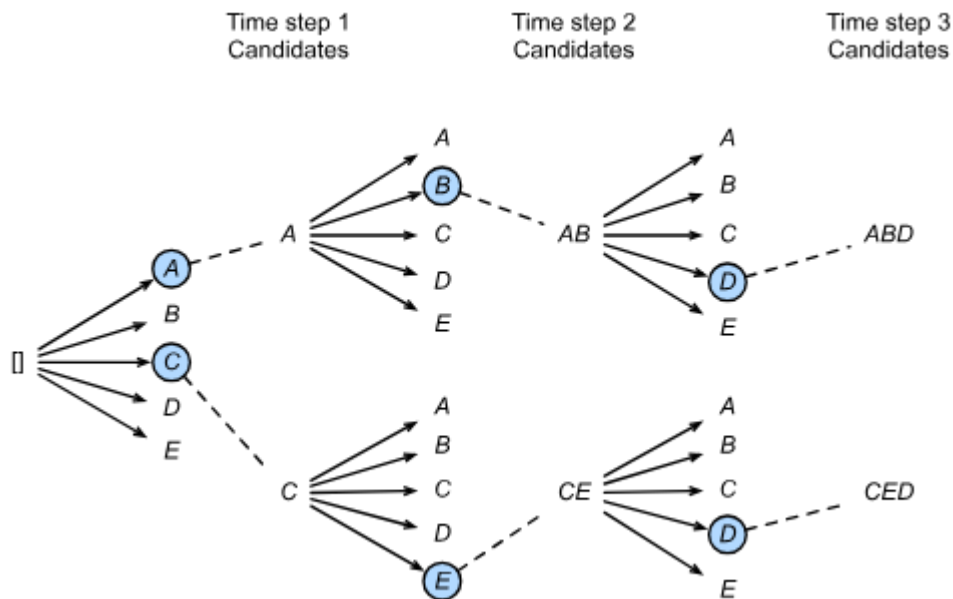
$$\text{score}(h_t, h_s) = h_t^T h_s$$

$$\alpha_t = \text{softmax}(\text{score})$$

$$\text{context}_t = \sum_s \alpha_{t,s} h_s$$

Vector ngữ cảnh context_t được kết hợp với output của Decoder để dự đoán từ tiếp theo.

2.4. Beam Search Decoding



Để cải thiện chất lượng dịch hơn, đồ án sử dụng thêm Beam Search Decoding . Beam Search sẽ duy trì đồng thời k chuỗi ứng viên tốt nhất tại mỗi bước thời gian. Mỗi chuỗi được mở rộng bằng cách xét nhiều khả năng từ tiếp theo, sau đó chọn ra k chuỗi có tổng log-xác suất cao nhất để tiếp tục.

3. Tiền xử lý dữ liệu

Tiền xử lý dữ liệu là bước quan trọng trước khi đưa vào mô hình Seq2Seq, nhằm chuẩn hóa văn bản thô và chuyển đổi nó thành dạng số có thể sử dụng trực tiếp cho quá trình huấn luyện và dự đoán. Cụ thể, các câu tiếng Anh và tiếng Pháp được chuẩn hóa về dạng chữ thường và loại bỏ các ký tự thừa. Sau đó, mỗi câu được token hóa thành các từ riêng biệt bằng tokenizer của Spacy, giúp mô hình nhận dạng các đơn vị cơ bản trong câu và học quan hệ giữa các từ trong câu nguồn và câu đích.

3.1. Tokenization

Trong bước tiền xử lý, dữ liệu tiếng Anh và tiếng Pháp được tách thành các token bằng công cụ spaCy tokenizer thông qua hàm get_tokenizer() của TorchText. Tokenizer có nhiệm vụ phân tách một câu thành các đơn vị nhỏ hơn (token), bao gồm từ và dấu câu. Không giống như việc tách bằng dấu cách thông thường, spaCy tokenizer xử lý tốt hơn các đặc trưng ngôn ngữ như dấu câu, từ ghép và ký tự đặc biệt.

Ví dụ:

Câu tiếng Anh: "A boy runs." => ["a", "boy", "runs", "."]

Câu tiếng Pháp: "Un garçon court." => ["un", "garçon", "court", "."]

Sau khi token hóa, mỗi token sẽ được ánh xạ sang một số nguyên tương ứng trong từ điển (vocabulary), giúp mô hình Seq2Seq nhận đầu vào dưới dạng số và dễ dàng xử lý. Việc token hóa giúp chuẩn hóa dữ liệu đầu vào, tạo điều kiện cho các bước tiếp theo như xây dựng từ điển và mã hóa câu sang dạng số để đưa vào mô hình Seq2Seq.

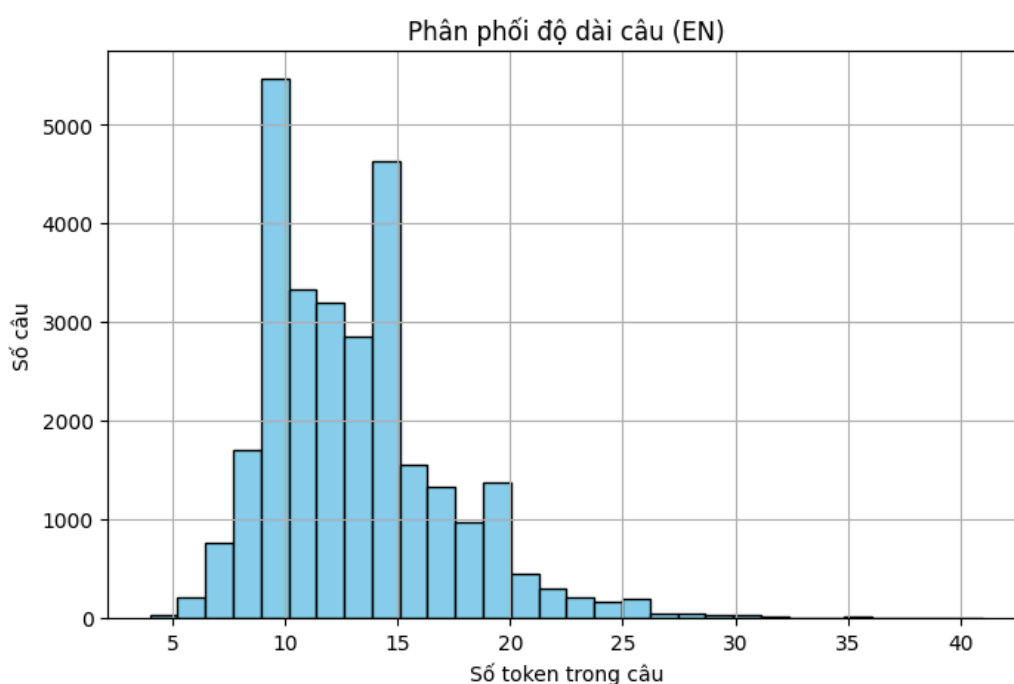
3.2. Xây dựng từ điển

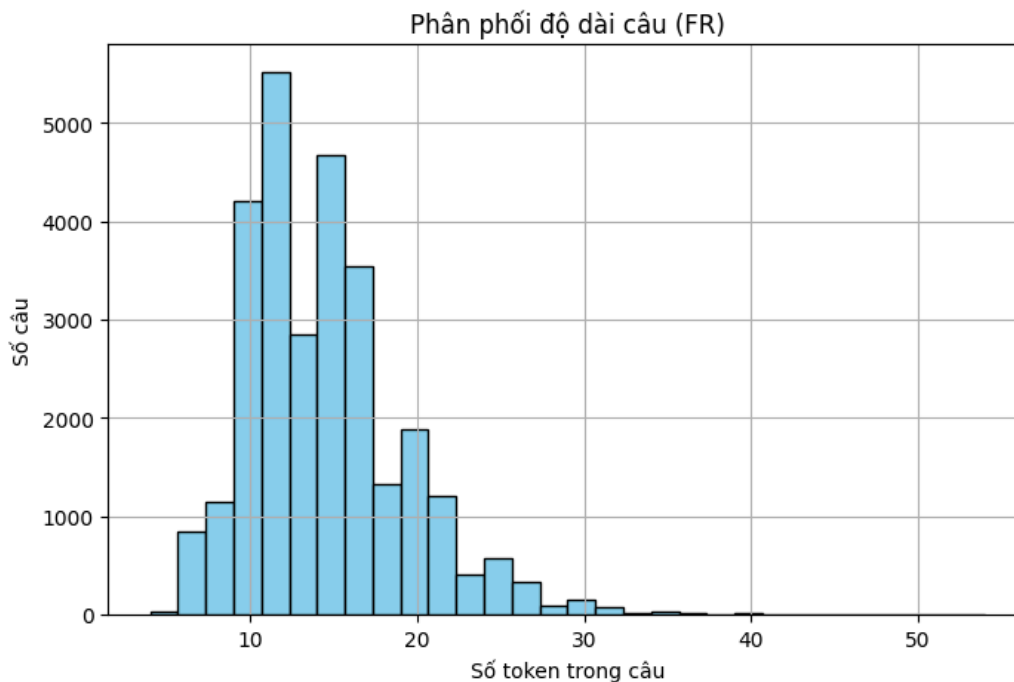
Sau khi đã token hóa dữ liệu, bước tiếp theo là tạo vocabulary (từ điển) cho từng ngôn ngữ. Mỗi file dữ liệu được đọc theo từng dòng, chuyển về chữ thường rồi tách thành các token bằng tokenizer của spaCy. Hàm `yield_tokens()` chỉ đơn giản là trả ra danh sách token của mỗi câu để dùng làm đầu vào cho quá trình xây vocab.

Từ các token này, hàm `build_vocab()` sẽ gom lại toàn bộ và tạo nên một danh sách các từ xuất hiện trong tập huấn luyện. Để tránh vocab quá lớn, số lượng từ được giới hạn ở một mức xác định. Ngoài ra, một vài token đặc biệt cũng được thêm vào, gồm `<unk>` (từ không có trong từ điển), `<pad>` (dùng để padding), `<sos>` (bắt đầu câu) và `<eos>` (kết thúc câu). Bất kỳ từ nào không nằm trong vocab sẽ được tự động chuyển về `<unk>`.

Vocabulary cho tiếng Anh và tiếng Pháp được tạo riêng dựa trên dữ liệu huấn luyện của từng ngôn ngữ. Sau cùng, chỉ số của token `<pad>` trong mỗi từ điển được lưu lại để sử dụng trong quá trình tạo batch và tính loss.

3.3. Padding & Packing





Trong các mô hình RNN hoặc LSTM, các câu trong tập dữ liệu thường có độ dài khác nhau, nhưng để huấn luyện theo batch, tất cả câu trong cùng một batch cần có cùng độ dài.

Padding là kỹ thuật thêm các token đặc biệt, thường là `<pad>` hoặc số 0, vào cuối các câu ngắn hơn để bằng độ dài câu dài nhất trong batch. Việc này cho phép mô hình xử lý nhiều câu cùng lúc, tăng hiệu quả tính toán trên GPU và tận dụng khả năng song song. Để tránh LSTM tính toán trên các token padding không có nghĩa, PyTorch cung cấp `pack_padded_sequence()`, giúp mô hình chỉ xử lý các token thực và bỏ qua `<pad>` trong quá trình forward pass, đồng thời đảm bảo tính toán chính xác và hiệu quả.

Packing là kỹ thuật giúp RNN/LSTM xử lý các câu có độ dài khác nhau một cách hiệu quả hơn sau khi đã padding. Mặc dù padding giúp các câu trong batch có cùng độ dài, các token `<pad>` không mang ý nghĩa và nếu LSTM xử lý chúng sẽ gây lãng phí tính toán và ảnh hưởng chất lượng mô hình. Hàm `pack_padded_sequence()` của PyTorch sẽ nén chuỗi đã padding, loại bỏ hoàn toàn các token `<pad>` và chỉ giữ lại các token thật của câu. Nhờ đó, LSTM chỉ xử lý đúng độ dài thực tế của từng câu, tránh tính toán thừa và tăng tốc độ huấn luyện. Khi cần, `pad_packed_sequence()` có thể giải nén chuỗi về lại dạng padded ban đầu.

3.4. DataLoader

Sau khi xây dựng dataset, dữ liệu được đưa vào DataLoader để chia thành các batch phục vụ cho quá trình huấn luyện. Hàm `get_dataloader()` nhận vào đường dẫn file tiếng Anh và tiếng Pháp, sau đó tạo một đối tượng `TranslationDataset`, nơi mỗi cặp câu sẽ được tokenize và chuyển sang dạng chỉ số dựa trên vocabulary đã xây dựng trước đó. Cuối cùng, DataLoader được tạo ra với kích thước batch cố định và có thể bật chế độ xáo trộn dữ liệu khi huấn luyện.

Điểm quan trọng trong DataLoader là hàm `collate_fn()`, dùng để xử lý nhiều mẫu trong cùng một batch: sắp xếp các câu theo độ dài, padding cho các câu ngắn hơn và trả về đúng dạng tensor mà mô hình LSTM yêu cầu. Nhờ đó, mỗi batch được đưa vào mô hình đều có kích thước thống nhất và phù hợp cho quá trình training và validation.

4. Xây dựng mô hình

Trong đồ án này chúng em triển khai 3 phiên bản mô hình Seq2Seq khác nhau để so sánh hiệu quả. Bao gồm: `BaseModel` (Encoder–Decoder thuần LSTM, không Attention), `Attention Model`: Thêm cơ chế Attention (Luong dot) vào Decoder, cuối cùng là `Beam Search Model` (Giữ nguyên mô hình Attention nhưng thay đổi chiến lược suy luận bằng Beam Search).

4.1. Encoder LSTM

Trong cả ba phiên bản mô hình triển khai, Encoder đều như nhau. Encoder có nhiệm vụ đọc câu tiếng Anh đầu vào và biến nó thành các vector mà Decoder có thể dùng để tạo câu dịch.

Đầu tiên, mỗi từ trong câu nguồn được ánh xạ sang một vector cố định bằng **Embedding Layer**. Sau đó, các vector này được đưa vào **LSTM nhiều tầng**. LSTM xử lý câu theo thứ tự từ trái sang phải và lưu lại thông tin ngữ cảnh của các từ đã đọc. Việc này giúp mô hình chuyển dữ liệu dạng văn bản thành dạng số có ý nghĩa, nơi các từ có quan hệ gần nhau trong ngữ nghĩa thì có vector gần nhau hơn.

Sau khi được embedding, chuỗi các vector này được đưa qua một **mạng LSTM nhiều tầng**. LSTM được dùng thay vì RNN thường vì khả năng ghi nhớ tốt hơn và giảm hiện tượng mất gradient. LSTM xử lý từng từ theo đúng thứ tự câu, đồng thời luôn mang theo thông tin ngữ cảnh từ những từ đã xuất hiện trước đó.

Kết quả đầu ra của Encoder gồm hai phần:

- **Chuỗi hidden states:** Đây là toàn bộ các vector ẩn tại mỗi bước thời gian. Chuỗi này đặc biệt quan trọng với mô hình có Attention, vì Attention sẽ dùng chúng để tính trọng số tập trung vào những từ liên quan trong câu nguồn.
- **Hidden state và Cell state cuối cùng:** Hai giá trị này được dùng để khởi tạo trạng thái ban đầu cho Decoder. Với mô hình Base (không có Attention), đây là nguồn thông tin chính mà Decoder dựa vào để sinh câu dịch.

Ở phiên bản Base Model, Decoder chỉ sử dụng hidden state cuối cùng. Còn với hai phiên bản Attention và Beam Search, toàn bộ chuỗi hidden states được tận dụng để tính toán trọng số Attention khi dịch từng từ.

Tóm lại, Encoder đóng vai trò như “bộ mã hoá ngữ cảnh đầu vào”, tạo ra biểu diễn vector cho cả câu, là nền tảng cho toàn bộ quá trình dịch trong các mô hình sau đó.

4.2. Decoder LSTM

Ở Decoder, mô hình sẽ tạo ra câu đích từng từ một. Mỗi bước dự đoán nhận vào từ đã tạo ở bước trước (hoặc từ thật trong dữ liệu nếu đang train với teacher forcing). Từ đó cũng được đưa qua lớp Embedding để chuyển thành vector.

Vector đầu vào này cùng với **trạng thái ẩn (hidden, cell)** mà Encoder trả về sẽ được đưa vào LSTM để mô hình quyết định từ tiếp theo cần sinh ra. Kết quả từ LSTM được đi qua một lớp tuyến tính để chuyển thành phân phối xác suất trên toàn bộ từ vựng. Từ có xác suất cao nhất sẽ được chọn làm đầu ra tại bước đó.

Quá trình này lặp lại cho tới khi mô hình sinh ra token `<eos>` hoặc đạt giới hạn độ dài. Ở bản có Attention thì Decoder còn dùng thêm thông tin từ Encoder, nhưng ở bản Base Model thì Decoder chỉ dựa trên hidden state và từ đầu vào của bước trước.

4.3. Attention (Luong dot)

Trong mô hình có Attention, Decoder không chỉ dựa vào hidden state mà Encoder trả về nữa. Thay vào đó, mỗi khi Decoder chuẩn bị sinh một từ mới, nó sẽ “quay lại” nhìn toàn bộ các hidden state mà Encoder tạo ra cho từng từ của câu nguồn.

Cơ chế Attention Luong (dot-product) mình dùng hoạt động khá đơn giản: Decoder lấy hidden state hiện tại của nó đem so khớp (dot product) với từng hidden của Encoder để xem chỗ nào “hợp lý” nhất. Những điểm nào khớp mạnh thì được gán

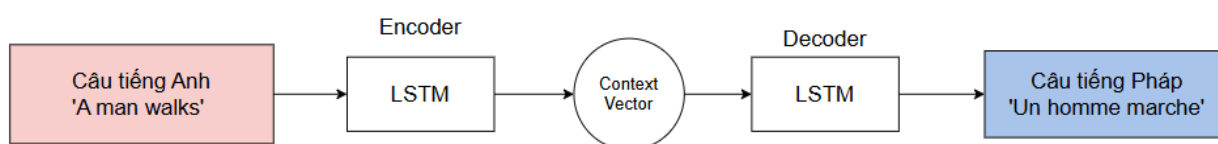
trọng số cao hơn. Sau khi chuẩn hóa bằng softmax, các trọng số này cho biết Decoder cần tập trung vào từ nào của câu nguồn.

Dựa trên các trọng số đó, mô hình tạo ra một vector ngữ cảnh (context). Hiểu đơn giản là: context này là phiên bản “tóm tắt thông minh” của cả câu, nhấn mạnh mạnh nhất vào những từ mà Decoder đang cần để đoán từ tiếp theo. Cuối cùng, Decoder ghép vector context với output LSTM rồi đi qua một lớp tuyến tính để sinh ra từ dự đoán.

Nhờ có Attention, mô hình xử lý được các câu dài tốt hơn, và Decoder không bị “quên” thông tin ở đầu câu như mô hình base.

5. Huấn luyện mô hình

5.1. Giới thiệu Baseline



Trong đồ án này, chúng em xây dựng một mô hình baseline làm cơ sở so sánh cho các phương pháp cải tiến ở các phần sau. Baseline được thiết kế dựa trên kiến trúc Seq2Seq Encoder–Decoder sử dụng LSTM, là mô hình tiêu chuẩn trong bài toán dịch máy Neural Machine Translation.

Mục tiêu của baseline là kiểm chứng khả năng ánh xạ câu nguồn sang câu đích trong điều kiện chưa áp dụng các kỹ thuật nâng cao như Attention hay Beam Search. Từ đó, baseline đóng vai trò làm mốc đánh giá để so sánh mức độ cải thiện khi tích hợp các phương pháp nâng cao.

Pipeline baseline bao gồm các bước cơ bản: tiền xử lý dữ liệu, huấn luyện mô hình Seq2Seq LSTM và đánh giá chất lượng dịch bằng BLEU Score. Mô hình baseline sử dụng phương pháp sinh chuỗi Greedy Decoding, trong đó tại mỗi bước thời gian, Decoder lựa chọn từ có xác suất cao nhất làm đầu ra.

5.2. Huấn luyện mô hình Baseline

Mô hình baseline được huấn luyện theo phương pháp học có giám sát (supervised learning) trên tập dữ liệu song ngữ Anh–Pháp. Trong quá trình huấn luyện, mô hình Seq2Seq LSTM học cách dự đoán từ tiếp theo trong câu đích dựa trên câu nguồn và các từ đã sinh trước đó.

Tham số	Giá trị	Giải thích
Embedding dimension	512	Kích thước vector biểu diễn mỗi từ sau bước embedding, giúp mô hình học được đặc trưng ngữ nghĩa phong phú của từ
Hidden dimension	512	Số chiều của trạng thái ẩn trong LSTM, quyết định khả năng ghi nhớ và biểu diễn thông tin chuỗi
Số tầng LSTM	2	Số lớp LSTM xếp chồng, giúp mô hình học được các đặc trưng ở nhiều mức trừu tượng
Dropout	0.3	Tỷ lệ dropout dùng để giảm overfitting trong quá trình huấn luyện
Batch size	64	Số lượng cặp câu nguồn–đích được xử lý trong mỗi bước huấn luyện
Số epoch tối đa	20	Số vòng lặp tối đa qua toàn bộ tập huấn luyện
Teacher forcing ratio	0.5	Xác suất sử dụng từ đúng làm đầu vào cho Decoder trong quá trình huấn luyện
Gradient clipping	1.0	Giới hạn chuẩn gradient nhằm tránh hiện tượng exploding gradient
Thiết bị huấn luyện	GPU / CPU	Mô hình được ưu tiên huấn luyện trên GPU nếu khả dụng

5.3. Quy trình huấn luyện mô hình

Quá trình huấn luyện mô hình được thực hiện theo từng batch và từng epoch.

Forward pass: Với mỗi batch dữ liệu, câu nguồn được đưa vào Encoder LSTM để tạo ra các trạng thái ẩn và trạng thái bộ nhớ. Decoder LSTM sinh câu đích từng token, bắt đầu từ token < sos >. Trong giai đoạn huấn luyện, teacher forcing được áp dụng với xác suất 0.5, nghĩa là tại mỗi bước thời gian, Decoder có thể sử dụng token đúng trong tập huấn luyện làm đầu vào thay vì token dự đoán trước đó. Cơ chế này giúp quá trình học hội tụ nhanh hơn và ổn định hơn.

Tính Loss: Hàm mất mát được sử dụng là Cross-Entropy Loss, được tính trên toàn bộ chuỗi đầu ra của Decoder, bỏ qua các token padding. Cross-Entropy Loss giúp

mô hình học phân phối xác suất đúng của các từ tiếp theo trong câu đích và là hàm mất mát tiêu chuẩn trong các bài toán dịch máy.

Backward pass và cập nhật trọng số: Sau khi tính loss, gradient được lan truyền ngược để cập nhật trọng số mạng. Gradient clipping với giới hạn 1.0 được áp dụng nhằm tránh hiện tượng exploding gradient, vốn là vấn đề phổ biến khi huấn luyện các mạng LSTM sâu.

Checkpoint: Sau mỗi epoch, nếu validation loss giảm so với giá trị tốt nhất trước đó, trạng thái của mô hình và optimizer sẽ được lưu lại. Cơ chế này cho phép tiếp tục huấn luyện nếu quá trình bị gián đoạn và đảm bảo mô hình tốt nhất được sử dụng cho dự đoán và đánh giá.

Early Stopping: Nếu validation loss không cải thiện trong một số epoch liên tiếp (patience = 3), quá trình huấn luyện sẽ dừng sớm. Cơ chế này giúp tránh overfitting và tiết kiệm thời gian huấn luyện.

6. Dự đoán

Sau khi mô hình Seq2Seq được huấn luyện, bước dự đoán nhằm sinh câu đích từ câu nguồn mới. Quá trình dự đoán dựa trên việc sử dụng Encoder để mã hóa câu nguồn thành các hidden states và cell states, chứa thông tin ngữ cảnh của toàn bộ câu. Decoder để sinh câu đích từng token một, dựa vào trạng thái ẩn hiện tại và thông tin ngữ cảnh từ Encoder. Và kết hợp cơ chế Attention để tập trung vào các phần quan trọng của câu nguồn. Quá trình dự đoán, chúng em sử dụng hai phương pháp phổ biến:

6.1. Greedy Decoding

Trong mô hình baseline, quá trình sinh câu đích được thực hiện bằng phương pháp Greedy Decoding. Tại mỗi bước thời gian, Decoder lựa chọn từ có xác suất cao nhất trong phân phối xác suất đầu ra để làm từ tiếp theo. Token được chọn này trở thành đầu vào cho bước tiếp theo của Decoder. Quá trình này tiếp tục cho đến khi sinh ra token <eos> hoặc đạt độ dài chuỗi tối đa (MAX_LEN_DECODING).

Greedy Decoding có ưu điểm là đơn giản và tốc độ suy luận nhanh, tuy nhiên dễ rơi vào nghiệm cục bộ và có thể bỏ lỡ các chuỗi dịch có xác suất tổng cao hơn.

6.2. Beam Search Decoding

Beam Search là phương pháp sinh chuỗi nâng cao được sử dụng trong các phiên bản cải tiến của mô hình. Thay vì chỉ giữ một chuỗi ứng viên như Greedy Decoding, Beam Search duy trì đồng thời n chuỗi ứng viên tốt nhất (beam) và mở rộng chúng bằng các token có xác suất tổng cao nhất. Khi token <eos> xuất hiện,

chuỗi đó được đưa vào danh sách hoàn tất. Sau khi đạt độ dài tối đa, chuỗi có tổng log-prob cao nhất (có thể chuẩn hóa theo chiều dài) được chọn làm kết quả cuối cùng. Nhờ đó, Beam Search giúp cải thiện chất lượng bản dịch, làm câu dịch tự nhiên và sát nghĩa hơn.

6.3 Dịch câu mới

Hàm `translate()` cho phép mô hình dịch bất kỳ câu tiếng Anh mới nào mà người dùng nhập vào. Câu được token hóa, mã hóa bằng Encoder, sinh câu đích bằng Decoder và trả về kết quả dưới dạng chuỗi tiếng Pháp. Hàm này có thể kết hợp với Greedy Decoding hoặc Beam Search, tùy vào nhu cầu về chất lượng và tốc độ dịch.

Đánh giá chất lượng dự đoán: Các câu đích sinh ra từ mô hình được đánh giá bằng BLEU score, so sánh với câu gốc trong tập kiểm tra. BLEU score cung cấp thước đo định lượng giúp so sánh chất lượng bản dịch giữa các phương pháp, giữa các phiên bản mô hình và đánh giá hiệu quả của Attention hay Beam Search trong cải thiện bản dịch.

7. Thảo luận

Trong quá trình triển khai mô hình Seq2Seq LSTM kết hợp Attention cho bài toán dịch máy Anh–Pháp, nhóm nhận thấy một số khó khăn và cơ hội cải tiến. Khi huấn luyện các câu dài, mô hình LSTM vẫn gặp hiện tượng gradient giảm dần, khiến việc học các phụ thuộc dài hạn trong câu trở nên khó khăn. Việc tinh chỉnh các tham số như kích thước embedding, hidden size, số layer và dropout cũng tốn nhiều thời gian thử nghiệm. Bên cạnh đó, những câu quá dài hoặc quá ngắn ảnh hưởng đến chất lượng dịch, đặc biệt khi sử dụng Greedy Decoding, bởi phương pháp này chỉ chọn từ có xác suất cao nhất tại mỗi bước sinh, đôi khi bỏ qua những lựa chọn tối ưu hơn ở các bước sau.

Để cải thiện chất lượng dịch trong tương lai, mô hình có thể được thay thế bằng kiến trúc Transformer, giúp học tốt các phụ thuộc dài hạn và nâng cao chất lượng dịch. Việc tăng lượng dữ liệu huấn luyện bằng cách sử dụng các corpus lớn hơn sẽ giúp mô hình học được nhiều cấu trúc ngữ pháp và từ vựng hơn. Ngoài ra, việc điều chỉnh beam width trong Beam Search có thể nâng cao chất lượng câu dịch, vì Beam Search giữ đồng thời nhiều ứng viên câu tốt nhất trong quá trình giải mã, giúp chọn ra câu dịch tổng thể tối ưu hơn so với Greedy Decoding, mặc dù chi phí tính toán sẽ cao hơn. Kết hợp các kỹ thuật tiền xử lý hiện đại như subword tokenization (BPE, WordPiece) cũng giúp giảm số từ OOV và cải thiện độ chính xác.

Quá trình dịch từng câu hoạt động theo cơ chế giải mã từ đầu đến cuối, trong đó Encoder mã hóa câu nguồn thành vector ngữ nghĩa, Decoder sinh từng từ dựa trên

vector này, và Attention giúp tập trung vào các từ quan trọng của câu nguồn. Cơ chế này giúp mô hình duy trì ý nghĩa gốc và cấu trúc ngữ pháp trong quá trình sinh câu dịch.

8. Kết luận

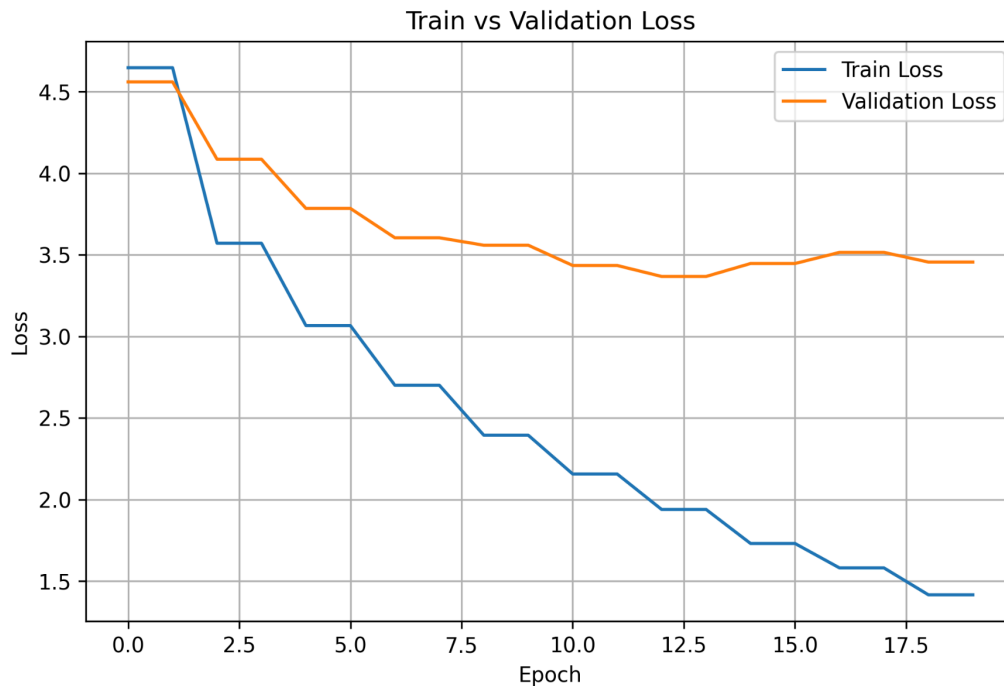
Dự án xây dựng mô hình dịch máy Anh–Pháp dựa trên kiến trúc Seq2Seq kết hợp Attention đã hoàn thành đầy đủ các mục tiêu đề ra, bao gồm tiền xử lý dữ liệu song ngữ, xây dựng từ điển, thiết kế encoder–decoder, tích hợp cơ chế Attention của Luong và đánh giá mô hình bằng BLEU. Kết quả thực nghiệm cho thấy mô hình có khả năng dịch tốt các câu ngắn và trung bình, tái hiện cấu trúc ngữ pháp cơ bản và giữ phần lớn ý nghĩa của câu nguồn. So với Greedy Decoding, Beam Search cải thiện chất lượng dịch rõ rệt, đặc biệt với các câu dài và phức tạp nhờ cơ chế giữ đồng thời nhiều ứng viên câu tốt nhất trong quá trình giải mã.

Mặc dù chất lượng dịch chưa thể sánh với các mô hình hiện đại như Transformer hay các mô hình ngôn ngữ lớn (LLM), dự án đã giúp nhóm hiểu sâu cơ chế hoạt động của Seq2Seq và Attention, những nền tảng quan trọng trong nhiều hệ thống dịch máy hiện nay. Những kiến thức và kinh nghiệm thu được sẽ là cơ sở vững chắc cho các nghiên cứu tiếp theo, đồng thời tạo tiền đề để triển khai các mô hình dịch máy với chất lượng cao hơn trong tương lai.

Thực nghiệm và đánh giá

1. Biểu đồ train/validation loss

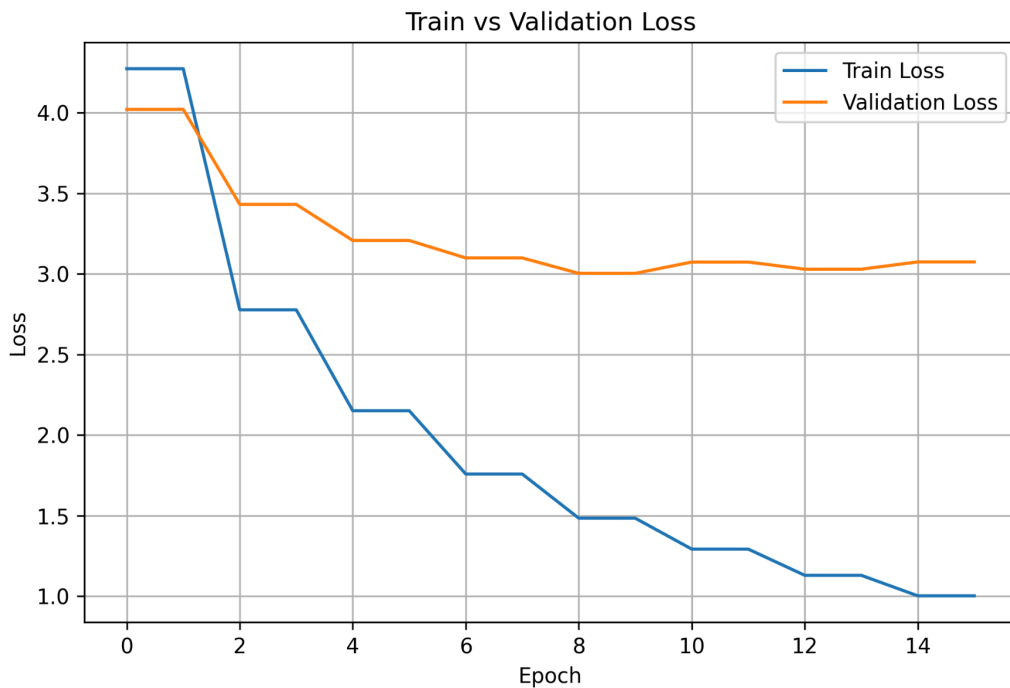
1.1. Biểu đồ mô hình cơ bản, chưa tích hợp Attention và Beam Search.



Biểu đồ thể hiện sự thay đổi của train loss và validation loss theo số epoch trong quá trình huấn luyện mô hình Seq2Seq LSTM cơ bản. Quan sát ta có thể thấy Train loss giảm nhanh, đều qua các epoch, cho thấy mô hình học tốt trên tập huấn luyện. Trong khi đó thì Validation Loss giảm chậm hơn và bắt đầu chững lại sau khoảng epoch thứ 7–8, đồng thời dao động nhẹ ở các epoch tiếp theo.

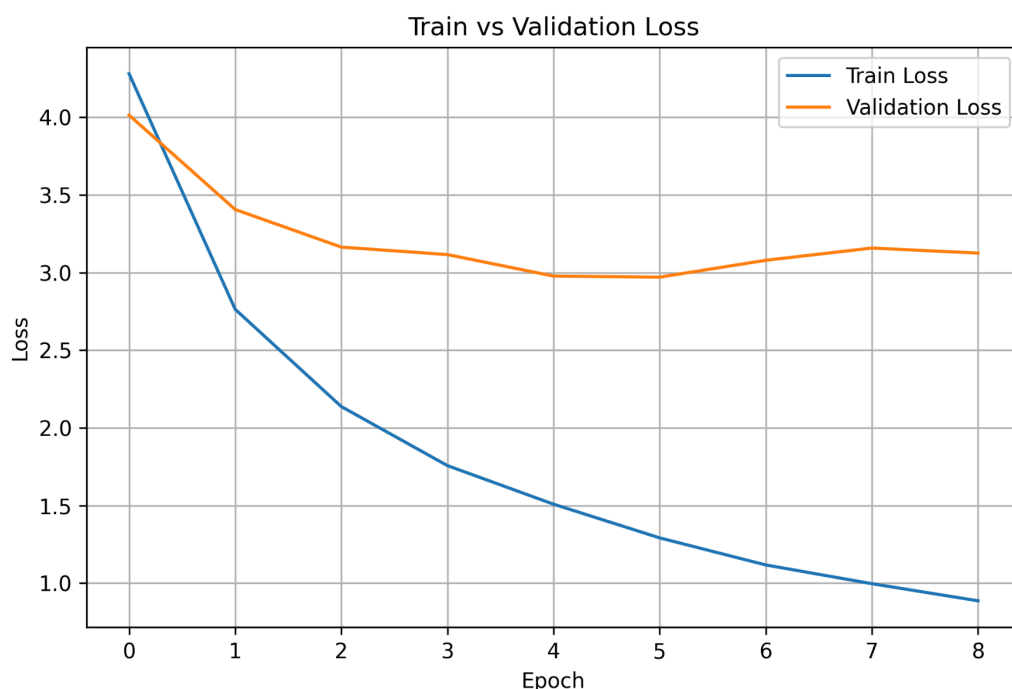
Từ biểu đồ ta thấy, mô hình có dấu hiệu overfitting nhẹ, khi khả năng tổng quát hóa trên tập validation không còn được cải thiện dù train loss tiếp tục giảm. Do đó, kỹ thuật early stopping được áp dụng để dừng huấn luyện tại thời điểm validation loss đạt giá trị tốt nhất.

1.2. Biểu đồ mô hình được tích hợp cơ chế Attention.



Biểu đồ cho thấy khi tích hợp cơ chế Attention, cả train loss và validation loss đều giảm nhanh hơn so với mô hình baseline. Đặc biệt, validation loss giảm rõ rệt trong các epoch đầu và đạt mức thấp hơn so với mô hình không Attention. Tuy nhiên, validation loss vẫn có xu hướng chững lại sau một số epoch, cho thấy mô hình vẫn còn dư địa cải thiện thêm.

1.3. Biểu đồ mô hình được tích hợp cơ chế Attention và Beam Search.



Biểu đồ cho thấy train loss giảm nhanh và đều theo số epoch, chứng tỏ mô hình học tốt trên tập huấn luyện khi được tích hợp cơ chế Attention. Validation loss giảm mạnh trong các epoch đầu và dần ổn định ở mức thấp hơn so với mô hình baseline, cho thấy khả năng tổng quát hóa của mô hình được cải thiện.

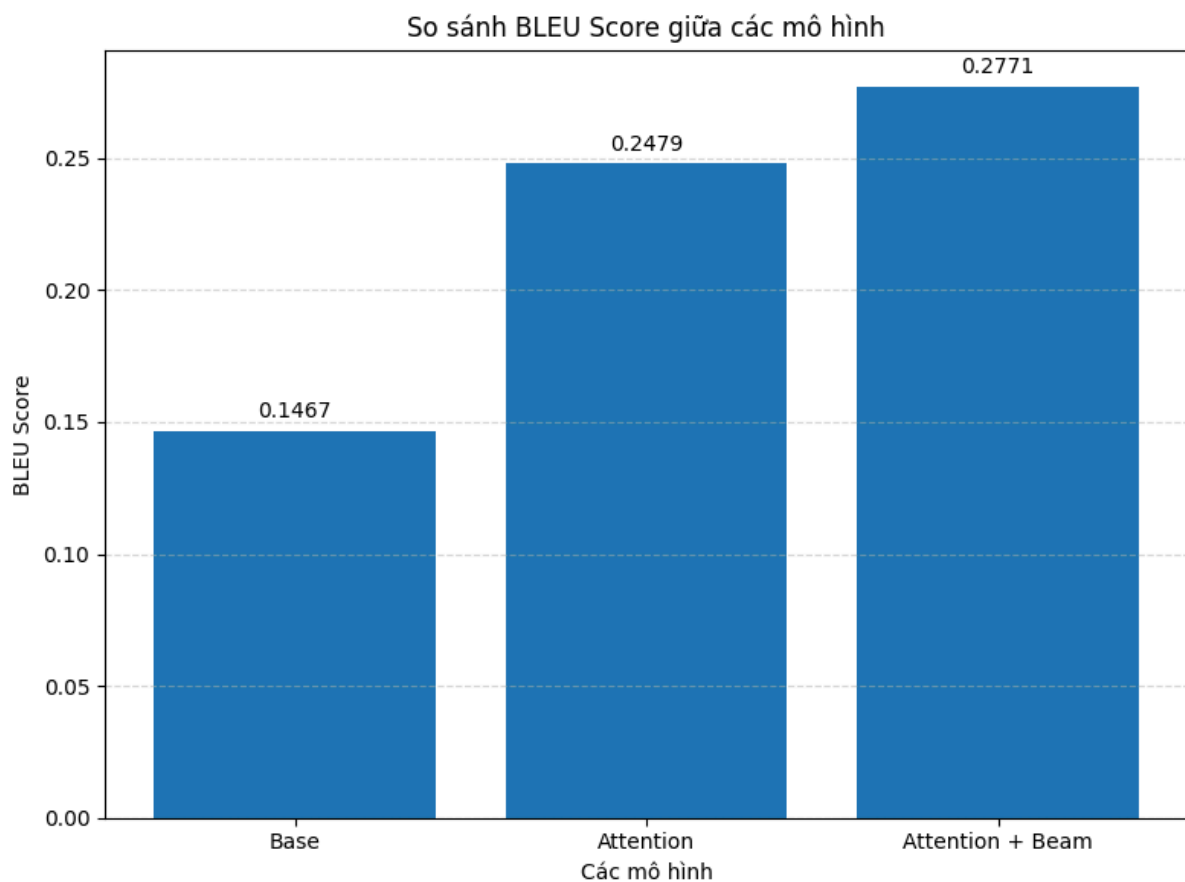
Mặc dù Beam Search không tác động trực tiếp đến quá trình huấn luyện, việc kết hợp Beam Search trong giai đoạn suy luận giúp mô hình lựa chọn chuỗi từ tối ưu hơn thay vì chỉ dựa vào xác suất cục bộ tại mỗi bước. Điều này góp phần cải thiện chất lượng bản dịch đầu ra, đặc biệt đối với các câu dài hoặc có cấu trúc phức tạp.

Khoảng cách giữa train loss và validation loss có xu hướng mở rộng nhẹ ở các epoch sau, tuy nhiên không xuất hiện hiện tượng overfitting nghiêm trọng. Kết quả này cho thấy mô hình đạt trạng thái hội tụ ổn định và phù hợp để sử dụng cho các bước đánh giá tiếp theo bằng BLEU score.

2. BLEU Score

Điểm đánh giá BLEU Score được tính dựa trên bộ test “test_2016_flickr.en” - “test_2016_flickr.fr” trong folder “data”

Mô hình	Mô tả	BLEU Score
Base Seq2Seq	Không Attention, không Beam Search	0.1467
Seq2Seq + Attention	Thêm cơ chế Attention	0.2479
Seq2Seq + Attention + Beam Search	Attention + Beam width = 3	0.2771



Nhận xét và phân tích kết quả:

Kết quả cho thấy mô hình Base Seq2Seq đạt BLEU Score thấp nhất. Phản ánh hạn chế của kiến trúc Seq2Seq truyền thống khi toàn bộ thông tin câu nguồn bị nén vào một context vector cố định. Khiến cho mô hình sẽ không thể dịch tốt khi gặp các câu dài hoặc có cấu trúc phức tạp.

Sau khi kết hợp thêm cơ chế Attention, ta thấy BLEU Score tăng lên một cách rõ rệt. Cơ chế Attention cho phép Decoder tập trung vào các từ quan trọng trong câu nguồn tại từng bước sinh từ, giúp cải thiện khả năng căn chỉnh giữa câu nguồn và câu đích, từ đó nâng cao chất lượng bản dịch.

Mô hình Seq2Seq kết hợp Attention và Beam Search đạt BLEU Score cao nhất là 0.2771. Việc sử dụng Beam Search (với beam width = 3) trong giai đoạn suy luận giúp mô hình xem xét nhiều chuỗi đầu ra tiềm năng thay vì chỉ chọn từ có xác suất cao nhất tại mỗi bước như Greedy Decoding. Nhờ đó, câu dịch sinh ra có tính mạch lạc và chính xác hơn.

Nói tóm lại, kết quả thực nghiệm cho thấy việc tích hợp cơ chế Attention giúp cải thiện rõ rệt chất lượng dịch, trong khi **Beam Search** tiếp tục giúp nâng cao BLEU Score ở mức vừa phải nhưng ổn định. Điều này chứng minh hiệu quả của các kỹ thuật cải tiến so với mô hình baseline ban đầu.

Kết quả dịch & Phân tích lỗi

Ví dụ 1: A man in an orange hat starring at something.

❖ **Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.**

Fr: un homme avec un chapeau orange regarde quelque chose.

Phân tích:

★ Điểm chính xác:

- **Dịch đúng cấu trúc ngữ pháp cơ bản:** Câu theo dạng *Chủ ngữ + động từ + tân ngữ*, phù hợp với câu gốc.
- **Cụm danh từ được chuyển nghĩa chính xác:** “A man in an orange hat” → “un homme avec un chapeau orange” → hoàn toàn tự nhiên và đúng nghĩa trong tiếng Pháp.
- **Giữ lại ý chính của câu:** Người đàn ông đang nhìn vào một vật gì đó → mô hình tái tạo được ý này.
- **Không mắc lỗi lặp từ hay rối ngữ pháp:** Một câu dịch sạch, rõ ràng, không có lỗi biểu hiện.

★ Điểm chưa chính xác:

- **Thiếu sắc thái của động từ “staring”:** *Staring* = “nhìn chăm chăm / nhìn chăm chú / nhìn lâu”, nhưng mô hình dịch thành “regarde” = “nhìn”, làm giảm độ mạnh của hành động.
- **Thiếu tính tự nhiên trong diễn đạt sắc thái:** Câu dịch đúng nghĩa nhưng chưa chọn từ hợp ngữ cảnh → không phản ánh hành động “nhìn chăm chú”.

Nhận xét: Đây là bản dịch khá tốt, gần đúng với nghĩa gốc. Thiếu duy nhất sắc thái mạnh của động từ.

❖ **Lần 2: Mô hình được tích hợp cơ chế Attention.**

Fr: un homme en chapeau orange regarde quelque chose.

Phân tích:

★ Điểm chính xác:

- **Chủ thể + hành động vẫn đúng** (“un homme... regarde quelque chose”).
- **Nghĩa tổng thể vẫn được giữ lại.**

★ Điểm chưa chính xác:

- **Sai về cụm mô tả trang phục:** “en chapeau orange” không tự nhiên và sai trong tiếng Pháp.
- **Không dịch được sắc thái “staring”** → vẫn chỉ dùng “regarde”.

Nhận xét: Kém hơn bản base do lỗi cấu trúc “en chapeau orange”.

❖ **Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.**

Fr: un homme en une casquette orange regarde quelque chose.

Phân tích:

★ Điểm chính xác:

- Chủ thể vẫn đúng (“un homme”).
- Động từ và tân ngữ vẫn đúng (“regarde quelque chose”).
- Ý tổng thể vẫn gần giống bản gốc.

★ Điểm chưa chính xác:

- **Cấu trúc sai hoàn toàn:** “*en une casquette orange*” → không tự nhiên, nghe sai trong tiếng Pháp.
- **Dùng từ “casquette”** không phù hợp bằng “chapeau” → “casquette” nghĩa là mũ lưỡi trai; câu gốc không nói rõ loại mũ.
- **Không dịch được sắc thái “staring”** (same issue).

Nhận xét: Kém nhất trong 3 bản, do lỗi cấu trúc nặng.

Ví dụ 2: A Boston Terrier is running on lush green grass in front of a white fence.

❖ **Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.**

Fr: un kitesurfeur tacheté se sur sur la pelouse devant une clôture blanche.

Phân tích:

★ Điểm chính xác:

- Cụm cuối “*devant une clôture blanche*” (trước hàng rào trắng) → đúng.

★ Điểm chưa chính xác:

- **Sai hoàn toàn chủ thể:** “Boston Terrier” → mô hình dịch thành “kitesurfeur tacheté” (một người kitesurfer bị đốm).
→ Sai loại thực thể (animal → human athlete).
- **Sai động từ:** không diễn đạt được “is running”.
- **Lỗi lặp từ:** “se sur sur”.
- **Lỗi ngữ pháp nặng:** cấu trúc câu sai.
- **Không dịch được** “lush green grass” (cỏ xanh tươi, abundant).
- **Không giữ được ý nghĩa tổng thể.**

Nhận xét: Dịch rất kém – diễn giải sai hoàn toàn chủ thể & hành động.

❖ Lần 2: Mô hình được tích hợp cơ chế Attention.

Fr: un patineur est sur un herbe vert vert devant devant une clôture blanche.

Phân tích:

★ Điểm chính xác:

- Dịch được cụm cuối: “devant une clôture blanche”
- Có cấu trúc thể hiện “*est sur*” → mô phỏng được “is ...ing” (dù sai động từ).

★ Điểm chưa chính xác:

- **Dịch sai chủ thể:** “Boston Terrier” → “patineur” = người trượt băng → vẫn sai.
- **Sai động từ “running”** → bị dịch thành “est sur” = “đang ở trên” → mất nghĩa “chạy”.
- **Lỗi lặp từ:** “vert vert”, “devant devant”.
- **Thiếu tự nhiên & sai ngữ pháp**
 - “un herbe” → đúng là “une herbe”.
 - “est sur un herbe vert vert” là sai cấu trúc.

Nhận xét: Dịch kém – có attention nhưng vẫn sai chủ thể, sai động từ, nhiều lỗi. Tốt hơn bản base một chút vì có cấu trúc câu hơn, nhưng vẫn rất sai.

❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.

Fr: un terrier terrier terrier court sur l' herbe verte devant une clôture blanche.

Phân tích:

★ Điểm chính xác:

- **Dịch đúng chủ thể gần như hoàn chỉnh:** “Boston Terrier” → “terrier” (Thiếu “Boston”, nhưng đúng loài động vật.)
- **Dịch đúng động từ:** “is running” → “court” → chính xác.
- **Dịch đúng địa điểm:** “on lush green grass” → “sur l'herbe verte” → sát nghĩa.
- **Dịch đúng cụm cuối:** “devant une clôture blanche”.
- **Cấu trúc câu tự nhiên:** Chủ ngữ → động từ → trạng ngữ → bổ ngữ.
- **Ý nghĩa tổng thể khớp hoàn toàn với câu gốc.**

★ Điểm chưa chính xác:

- **Lặp từ “terrier terrier terrier”** → Đây là lỗi repetition do beam search quá ưu tiên các từ có xác suất cao.
- **Thiếu tính từ “Boston”** nhưng vẫn nhận diện đúng rằng đây là một giống chó Terrier.

Nhận xét: Bản dịch sát nghĩa nhất, đúng chủ thể, đúng động từ, ngữ cảnh không gian, tự nhiên hơn nhiều, chỉ có lỗi lặp từ. Nhưng tốt hơn hẳn so với 2 bản trước.

Ví dụ 3: A girl in karate uniform breaking a stick with a front kick.

❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.

Fr: une fille en tenue de karaté avec un bâton avec un un.

Phân tích:

★ Điểm chính xác:

- Dịch đúng phần đầu: “une fille en tenue de karaté” → chính xác và tự nhiên.
- Nhận ra chủ thể và trang phục.

★ Điểm chưa chính xác:

- Không dịch được hành động “breaking a stick”.
- Không dịch được “with a front kick” → không hiện ra động tác đá.
- Lặp từ, lỗi cú pháp nặng: “avec un bâton avec un un”.
- Thiếu động từ chính, thiếu cấu trúc câu hoàn chỉnh.

Nhận xét: Dịch rất kém – chỉ dịch được phần mô tả đầu câu.

❖ Lần 2: Mô hình được tích hợp cơ chế Attention.

Fr: une fille en tenue de karaté s' un bâton avec un coup de pied.

Phân tích:

★ Điểm chính xác:

- Vẫn giữ đúng cụm: “une fille en tenue de karaté”.
- Một phần ý nghĩa của “front kick” → “*coup de pied*” xuất hiện (dịch thành cú đá).

★ Điểm chưa chính xác:

- **Lỗi nghiêm trọng ở phần động từ:** “s' un bâton” → hoàn toàn sai, không có nghĩa.
- Không dịch đúng “breaking a stick” (bị mất nghĩa bẻ/đá gãy).
- Trật tự câu lộn xộn, không đúng cú pháp.
- Thiếu động từ mô tả hành động.

Nhận xét: Tốt hơn bản base một chút, nhưng vẫn sai nhiều và không hiểu được hành động chính.

❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.

Fr: une fille en tenue de karaté frappant un coup avec un bâton de pied.

Phân tích:

★ Điểm chính xác:

- **Cụm chủ thể + trang phục đúng:** “une fille en tenue de karaté”.
- **Có động từ mô tả hành động:** “*frappant un coup*” (đang đánh/ra đòn).
- **Có ý nghĩa về cú đá:** từ “coup” xuất hiện mang sắc thái hành động.

★ Điểm chưa chính xác:

- Cấu trúc sai và vô nghĩa, chưa dịch chính xác hành động chính.

Nhận xét: Tốt nhất trong 3 bản, vì có động từ mô tả hành động, câu có cấu trúc hơn và gần đúng một phần ý nghĩa.

Ví dụ 4: Five people wearing winter jackets and helmets stand in the snow, with snowmobiles in the background.

❖ **Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.**

Fr: cinq personnes portant des casques de soleil et des vestes , des sacs de soleil , avec des en arrière-plan.

Phân tích:

★ Điểm chính xác:

- “cinq personnes” → đúng số lượng.
- Có nhận ra cấu trúc “portant ... et ...” → câu có dạng mô tả.

★ Điểm chưa chính xác:

- **Sai hoàn toàn về từ vựng trang phục:**
 - “casques de soleil” = *mũ chống nắng* → sai hoàn toàn so với “helmets”.
 - “vestes” không mang nghĩa “winter jackets” → thiếu sắc thái mùa đông.
 - “sacs de soleil” = *túi nắng* → hoàn toàn vô nghĩa.
- **Mất thông tin quan trọng:** Không có “neige” (tuyết), không có động từ “stand” (đứng).
- **Cụm cuối “avec des en arrière-plan”** → sai ngữ pháp + thiếu danh từ chính (không có “snowmobiles”).

Nhận xét: Dịch rất kém – ý nghĩa gần như bị sai toàn bộ.

❖ **Lần 2: Mô hình được tích hợp cơ chế Attention.**

Fr: cinq personnes portant des manteaux d'hiver et des casques sont debout dans la neige , avec des buissons en arrière-plan.

Phân tích:

★ Điểm chính xác:

- Dịch chính xác “winter jackets” → *manteaux d’hiver*.
- Dịch chính xác “helmets” → *casques*.
- Dịch chính xác “stand in the snow” → *sont debout dans la neige*.
- Cấu trúc câu tự nhiên, rõ ràng.

★ Điểm chưa chính xác:

- Dịch sai “snowmobiles” → thành “buissons” (bụi cây).
- Về mặt ý nghĩa thì “buissons” không liên quan gì đến phương tiện snowmobile.

Nhận xét: Dịch khá tốt, sai duy nhất phần cuối.

❖ Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.

Fr: cinq personnes portant des manteaux d'hiver et des casques sont debout dans la neige , avec des voiliers en arrière-plan.

Phân tích:

★ Điểm chính xác:

- Toàn bộ phần đầu giống bản Attention → đúng và tự nhiên.
- Cấu trúc tốt, dễ đọc.

★ Điểm chưa chính xác:

- “voiliers” = **thuyền buồm** → hoàn toàn không liên quan đến snowmobiles.

Nhận xét: Đúng y hệt bản Attention, chỉ sai danh từ cuối.

Ví dụ 5: People are fixing the roof of a house.

❖ Lần 1: Huấn luyện mô hình Seq2Seq cơ bản, chưa tích hợp Attention và Beam Search.

Fr: des gens sont à la fenêtre d'une maison.

Phân tích:

★ Điểm chính xác:

- “des gens” → đúng cụm “People”.
- “d’une maison” → đúng phần “of a house”.

★ Điểm chưa chính xác:

- “sont à la fenêtre” = **đang ở cửa sổ** → Sai hoàn toàn với ý “fixing the roof”.
- Không có động từ “réparer” hoặc ý nghĩa sửa chữa.
- Sai vị trí: roof ≠ fenêtre → Lỗi nghiêm trọng về nghĩa.

Nhận xét: Sai hoàn toàn hành động và vị trí → Chất lượng dịch rất kém.

❖ **Lần 2: Mô hình được tích hợp cơ chế Attention.**

Fr: des gens réparent le moteur d'une maison.

Phân tích:

★ Điểm chính xác:

- “des gens réparent” → đúng hành động sửa chữa (fixing).
- Giữ được cấu trúc câu đúng.

★ Điểm chưa chính xác:

- **Sai hoàn toàn đối tượng:** “le moteur d’une maison” = *động cơ của một ngôi nhà* → vô thực, không tồn tại.
- Không phản ánh được từ “roof”.
- Mặc dù dịch đúng động từ, nhưng sai đối tượng khiến câu dịch không đúng ý.

Nhận xét: Tốt hơn bản ban đầu, nhưng vẫn sai nặng vì nhận sai vật thể cần sửa.

❖ **Lần 3: Mô hình được tích hợp cơ chế Attention và Beam Search.**

Fr: des gens réparent le toit d'une maison.

Phân tích:

★ Điểm chính xác:

- “des gens réparent” → đúng hành động.
- “le toit d’une maison” → dịch chính xác 100% cụm “the roof of a house”.
- Câu đúng ngữ pháp, tự nhiên, rõ ràng.
- Bám sát nghĩa của câu gốc nhất.

★ Điểm chưa chính xác:

- Không có lỗi ngữ pháp hay nghĩa đáng kể.

Nhận xét: Dịch chính xác, tự nhiên, đúng hoàn toàn nội dung gốc. Đây là bản dịch tốt nhất.

→ **Kết luận:** Qua kết quả dịch và phân tích lỗi trên 5 câu thử nghiệm, có thể thấy:

- Mô hình Base Seq2Seq cho chất lượng thấp nhất, thường dịch sai chủ thể và hành động do không duy trì được ngữ cảnh.
- Mô hình có Attention cải thiện rõ rệt khi nhận diện chính xác hơn các cụm mô tả và cấu trúc câu, dù vẫn còn xuất hiện các lỗi chọn sai danh từ.
- Mô hình Attention kết hợp Beam Search cho kết quả tốt nhất, tạo ra các câu dịch tự nhiên và sát nghĩa hơn nhờ khả năng tìm chuỗi từ tối ưu.

- Tuy nhiên, cả ba mô hình vẫn gặp khó khăn với các chi tiết ngữ nghĩa tinh tế (như sắc thái động từ hoặc danh từ chuyên biệt), cho thấy giới hạn của kiến trúc Seq2Seq dựa trên LSTM trong các tác vụ dịch phức tạp.
- Nhìn chung, kết quả cho thấy việc tích hợp Attention và Beam Search là cần thiết để nâng cao chất lượng dịch, và là nền tảng quan trọng trước khi tiến tới các mô hình dịch hiện đại hơn.

Phụ lục

Link code, data, checkpoint:  **NLP Sources**

Mã nguồn:

1. Cài đặt môi trường và tiền xử lý dữ liệu:

1.1. Import thư viện:

```
import os
import random
import time

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence,
pad_packed_sequence
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator

from nltk.translate.bleu_score import sentence_bleu,
SmoothingFunction
import matplotlib.pyplot as plt
```

1.2. Cấu hình tham số:

```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
BATCH_SIZE = 64
EMBED_DIM = 512
HIDDEN_DIM = 512
NUM_LAYERS = 2
DROPOUT = 0.3
N_EPOCHS = 20
CLIP = 1.0
TEACHER_FORCING_RATIO = 0.5
MAX_LEN_DECODING = 50
CHECKPOINT_DIR = './checkpoints'
os.makedirs(CHECKPOINT_DIR, exist_ok=True)

EN_TRAIN = './data/train.en'
FR_TRAIN = './data/train.fr'
EN_VAL = './data/val.en'
FR_VAL = './data/val.fr'
EN_TEST = './data/test_2016_flickr.en'
FR_TEST = './data/test_2016_flickr.fr'
```

1.3. Tokenization:

```
en_tokenizer = get_tokenizer('spacy', language='en_core_web_sm')
fr_tokenizer = get_tokenizer('spacy', language='fr_core_news_sm')
```

1.4. Xây dựng từ điển:

```
def yield_tokens(file_path, tokenizer):
    with open(file_path, encoding='utf-8') as f:
        for line in f:
            yield tokenizer(line.strip().lower())

def build_vocab(file_path, tokenizer, max_tokens=10000):
    vocab = build_vocab_from_iterator(
        yield_tokens(file_path, tokenizer),
        specials=['<unk>', '<pad>', '<sos>', '<eos>'],
        max_tokens=max_tokens
    )
    vocab.set_default_index(vocab['<unk>'])
    return vocab
```

```
en_vocab = build_vocab(EN_TRAIN, en_tokenizer)
fr_vocab = build_vocab(FR_TRAIN, fr_tokenizer)
```

```
PAD_IDX_EN = en_vocab['<pad>']
PAD_IDX_FR = fr_vocab['<pad>']
```

1.5. Hàm chuyển câu thành chuỗi token ID:

```
def encode_sentence(sentence, tokenizer, vocab):
    tokens = tokenizer(sentence.strip().lower())
    ids = [vocab['<sos>']] + [vocab[token] for token in tokens] +
[vocab['<eos>']]
    return torch.tensor(ids, dtype=torch.long)
```

1.6. Translation Dataset:

```
class TranslationDataset(Dataset):
    def __init__(self, src_file, trg_file, src_tokenizer,
trg_tokenizer, src_vocab, trg_vocab):
        with open(src_file, encoding='utf-8') as f:
            self.src_lines = [l.strip() for l in f.readlines()]
        with open(trg_file, encoding='utf-8') as f:
            self.trg_lines = [l.strip() for l in f.readlines()]
        assert len(self.src_lines) == len(self.trg_lines)
        self.src_tokenizer = src_tokenizer
        self.trg_tokenizer = trg_tokenizer
        self.src_vocab = src_vocab
        self.trg_vocab = trg_vocab
```

```

def __len__(self):
    return len(self.src_lines)

def __getitem__(self, idx):
    src = encode_sentence(self.src_lines[idx],
self.src_tokenizer, self.src_vocab)
    trg = encode_sentence(self.trg_lines[idx],
self.trg_tokenizer, self.trg_vocab)
    return src, trg

```

1.7. Collate fn với Padding & Packing:

```

def collate_fn(batch):
    src_batch, trg_batch = zip(*batch)
    src_lengths = torch.tensor([len(s) for s in src_batch],
dtype=torch.long)
    trg_lengths = torch.tensor([len(t) for t in trg_batch],
dtype=torch.long)

    sorted_idx = torch.argsort(src_lengths, descending=True)
    src_batch = [src_batch[i] for i in sorted_idx]
    trg_batch = [trg_batch[i] for i in sorted_idx]
    src_lengths = src_lengths[sorted_idx]
    trg_lengths = trg_lengths[sorted_idx]

    src_padded = pad_sequence(src_batch, batch_first=True,
padding_value=PAD_IDX_EN)
    trg_padded = pad_sequence(trg_batch, batch_first=True,
padding_value=PAD_IDX_FR)

    return src_padded, trg_padded, src_lengths, trg_lengths

```

1.8. DataLoader:

```

def get_dataloader(src_file, trg_file, batch_size=BATCH_SIZE,
shuffle=True):
    dataset = TranslationDataset(src_file, trg_file, en_tokenizer,
fr_tokenizer, en_vocab, fr_vocab)
    return DataLoader(dataset, batch_size=batch_size,
shuffle=shuffle, collate_fn=collate_fn)

```

2. Xây dựng mô hình:

2.1. Encoder:

```

class Encoder(nn.Module):
    def __init__(self, input_dim, embed_dim, hidden_dim,
num_layers, dropout, pad_idx):
        super().__init__()

```

```

        self.embedding = nn.Embedding(input_dim, embed_dim,
padding_idx=pad_idx)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
num_layers=num_layers, dropout=dropout, batch_first=True)

    def forward(self, src, src_lengths):
        embedded = self.embedding(src)
        packed_emb = pack_padded_sequence(embedded,
src_lengths.cpu(), batch_first=True, enforce_sorted=True)
        packed_outputs, (hidden, cell) = self.lstm(packed_emb)
        outputs, _ = pad_packed_sequence(packed_outputs,
batch_first=True, padding_value=0.0)
        return outputs, hidden, cell

```

2.2. Attention:

```

class LuongAttention(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, decoder_hidden, encoder_outputs,
mask=None):
        scores = torch.bmm(encoder_outputs,
decoder_hidden.unsqueeze(2)).squeeze(2)
        if mask is not None:
            scores = scores.masked_fill(~mask, -1e9)
        attn_weights = torch.softmax(scores, dim=1)
        context = torch.bmm(attn_weights.unsqueeze(1),
encoder_outputs).squeeze(1)
        return context, attn_weights

```

2.3. Decoder:

```

class Decoder(nn.Module):
    def __init__(self, output_dim, embed_dim, hidden_dim,
num_layers, dropout, pad_idx):
        super().__init__()
        self.embedding = nn.Embedding(output_dim, embed_dim,
padding_idx=pad_idx)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
num_layers=num_layers, dropout=dropout, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim*2, output_dim)
        self.attention = LuongAttention()

    def forward(self, input_token, hidden, cell, encoder_outputs,
src_mask=None):
        input_token = input_token.unsqueeze(1)
        embedded = self.embedding(input_token)

```



```

        output, (hidden, cell) = self.lstm(embedded, (hidden,
cell))
        dec_hidden = hidden[-1]
        context, attn_weights = self.attention(dec_hidden,
encoder_outputs, mask=src_mask)
        concat = torch.cat([output.squeeze(1), context], dim=1)
        prediction = self.fc_out(concat)
        return prediction, hidden, cell, attn_weights

```

2.4. Seq2Seq:

```

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device,
teacher_forcing_ratio=TEACHER_FORCING_RATIO):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
        self.teacher_forcing_ratio = teacher_forcing_ratio

    def forward(self, src, src_lengths, trg=None,
teacher_forcing=True):
        batch_size = src.size(0)
        trg_len = trg.size(1) if trg is not None else
MAX_LEN_DECODING
        output_dim = self.decoder.fc_out.out_features
        outputs = torch.zeros(batch_size, trg_len,
output_dim).to(self.device)
        encoder_outputs, hidden, cell = self.encoder(src,
src_lengths)
        src_mask = (src != PAD_IDX_EN).to(self.device)
        input_token = trg[:,0] if trg is not None else
torch.tensor([fr_vocab['<sos>']]*batch_size, device=self.device)
        for t in range(1, trg_len):
            teacher_force_flag = (random.random() <
self.teacher_forcing_ratio) if (trg is not None and
teacher_forcing) else False
            output, hidden, cell, _ = self.decoder(input_token,
hidden, cell, encoder_outputs, src_mask)
            outputs[:, t] = output
            top1 = output.argmax(1)
            input_token = trg[:,t] if teacher_force_flag and trg
is not None else top1
        return outputs

```

3. Huấn luyện mô hình:

3.1. Hàm huấn luyện một epoch:

```
def train_one_epoch(model, dataloader, optimizer, criterion, clip,
device):
    model.train()
    epoch_loss = 0
    for src, trg, src_lengths, trg_lengths in dataloader:
        src, trg, src_lengths = src.to(device), trg.to(device),
src_lengths.to(device)
        optimizer.zero_grad()
        output = model(src, src_lengths, trg,
teacher_forcing=True)
        output_dim = output.shape[-1]
        output_flat = output[:,1:,:].reshape(-1, output_dim)
        trg_flat = trg[:,1:].reshape(-1)
        loss = criterion(output_flat, trg_flat)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
        optimizer.step()
        epoch_loss += loss.item()
    return epoch_loss / len(dataloader)
```

3.2. Hàm đánh giá mô hình trên tập validation:

```
def evaluate(model, dataloader, criterion, device):
    if dataloader is None:
        return float('inf')
    model.eval()
    epoch_loss = 0
    with torch.no_grad():
        for src, trg, src_lengths, trg_lengths in dataloader:
            src, trg, src_lengths = src.to(device),
trg.to(device), src_lengths.to(device)
            output = model(src, src_lengths, trg,
teacher_forcing=False)
            output_dim = output.shape[-1]
            output_flat = output[:,1:,:].reshape(-1, output_dim)
            trg_flat = trg[:,1:].reshape(-1)
            loss =
nn.CrossEntropyLoss(ignore_index=PAD_IDX_FR)(output_flat,
trg_flat)
            epoch_loss += loss.item()
    return epoch_loss / len(dataloader)
```

3.3. Hàm huấn luyện toàn bộ mô hình và in ra biểu đồ train/val loss:

```
def run_training():
```

```

    enc = Encoder(len(en_vocab), EMBED_DIM, HIDDEN_DIM,
NUM_LAYERS, DROPOUT, PAD_IDX_EN)
    dec = Decoder(len(fr_vocab), EMBED_DIM, HIDDEN_DIM,
NUM_LAYERS, DROPOUT, PAD_IDX_FR)
    model = Seq2Seq(enc, dec, DEVICE,
TEACHER_FORCING_RATIO).to(DEVICE)

    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX_FR)

    train_loader = get_dataloader(EN_TRAIN, FR_TRAIN,
batch_size=BATCH_SIZE, shuffle=True)
    val_loader = get_dataloader(EN_VAL, FR_VAL,
batch_size=BATCH_SIZE, shuffle=False) if os.path.exists(EN_VAL)
else None

    best_valid_loss = float('inf')
    patience = 3
    wait = 0

    # === Thêm biến lưu loss ===
    train_losses = []
    valid_losses = []
    logs = []

    for epoch in range(1, N_EPOCHS+1):
        start = time.time()

        train_loss = train_one_epoch(model, train_loader,
optimizer, criterion, CLIP, DEVICE)
        valid_loss = evaluate(model, val_loader, criterion,
DEVICE) if val_loader else train_loss
        end = time.time()
        duration = end - start

        # === Lưu loss mỗi epoch ===
        train_losses.append(train_loss)
        valid_losses.append(valid_loss)

        print(f"Epoch {epoch} | Train Loss: {train_loss:.4f} | Val
Loss: {valid_loss:.4f} | Time: {duration:.1f}s")

    # === Lưu vào bảng log ===
    logs.append({
        "epoch": epoch,
        "train_loss": train_loss,
        "val_loss": valid_loss,
        "time_sec": round(duration, 2)
    })

```

```

    })

    # Save best checkpoint
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'valid_loss': valid_loss
        }, os.path.join(CHECKPOINT_DIR, 'best_model.pth'))
        wait = 0
        print("    Saved new best model.")
    else:
        wait += 1
        if wait >= patience:
            print("Early stopping triggered.")
            break

    # ===== In bảng log =====
    print("\n===== Training Logs =====")
    print(f"{'Epoch':<7} {'Train Loss':<12} {'Val Loss':<12} {'Time (s)':<10}")
    print("-" * 45)

    for row in logs:
        print(f"{'epoch':<7} {'train_loss':<12.4f} {'val_loss':<12.4f} {'time_sec':<10}")

    # === Vẽ biểu đồ train/val loss ===
    plt.figure(figsize=(8,5))
    plt.plot(train_losses, label='Train Loss')
    plt.plot(valid_losses, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Train vs Validation Loss')
    plt.legend()
    plt.grid(True)
    plt.savefig('Loss_curve.png', dpi=300)
    plt.close()

    print("Đã lưu biểu đồ loss.")

    return model

```

4. Dự đoán:

4.1. Greedy Decoding:

```
def translate(sentence: str, model, device,
max_len=MAX_LEN_DECODING):
    model.eval()
    src_tensor = encode_sentence(sentence, en_tokenizer,
en_vocab).unsqueeze(0).to(device)
    src_lengths = torch.tensor([src_tensor.size(1)],
dtype=torch.long).to(device)

    with torch.no_grad():
        encoder_outputs, hidden, cell = model.encoder(src_tensor,
src_lengths)

        sos_idx = fr_vocab['<sos>']
        eos_idx = fr_vocab['<eos>']
        input_token = torch.tensor([sos_idx], dtype=torch.long,
device=device)

        src_mask = (src_tensor != PAD_IDX_EN).to(device)

        output_ids = []
        for _ in range(max_len):
            with torch.no_grad():
                pred, hidden, cell, attn = model.decoder(input_token,
hidden, cell, encoder_outputs, src_mask)
                top1 = pred.argmax(1).item()
                if top1 == eos_idx:
                    break
                output_ids.append(top1)
                input_token = torch.tensor([top1], dtype=torch.long,
device=device)

        # detokenize
        try:
            itos = fr_vocab.get_itos()
        except:
            itos = [tok for tok, idx in
sorted(fr_vocab.get_stoi().items(), key=lambda x: x[1])]
        words = [itos[i] for i in output_ids]
        return " ".join(words)
```

4.2. Beam Search Decoding:

```
def beam_translate(sentence: str, model, device,
max_len=MAX_LEN_DECODING, beam_width=3):
    model.eval()
```

```

    src_tensor = encode_sentence(sentence, en_tokenizer,
en_vocab).unsqueeze(0).to(device)
    src_lengths = torch.tensor([src_tensor.size(1)],
device=device)
    with torch.no_grad():
        encoder_outputs, hidden, cell = model.encoder(src_tensor,
src_lengths)

    sos_idx, eos_idx = fr_vocab['<sos>'], fr_vocab['<eos>']
    src_mask = (src_tensor != PAD_IDX_EN).to(device)

    # beam: list of (tokens, hidden, cell, score)
    beam = [( [sos_idx], hidden, cell, 0.0 )]
    completed = []

    for _ in range(max_len):
        new_beam = []
        for tokens, h, c, score in beam:
            if tokens[-1] == eos_idx:
                completed.append((tokens, score)) # chỉ lưu
tokens + score
                continue
            input_token = torch.tensor([tokens[-1]],
device=device)
            with torch.no_grad():
                pred, h_new, c_new, _ = model.decoder(input_token,
h, c, encoder_outputs, src_mask)
                log_probs = F.log_softmax(pred, dim=1)
                topk_logp, topk_idx = torch.topk(log_probs,
beam_width, dim=1)
                for i in range(beam_width):
                    new_beam.append((tokens+[topk_idx[0,i].item()],
h_new, c_new, score+topk_logp[0,i].item()))
            beam = sorted(new_beam, key=lambda x: x[3],
reverse=True)[:beam_width]
            if not beam:
                break

    # thêm những beam còn lại nếu chưa gặp <eos>
    for tokens, h, c, score in beam:
        completed.append((tokens, score))

    # chọn best
    best_tokens, _ = max(completed, key=lambda x: x[1]/len(x[0]))
    output_ids = best_tokens[1:] # bỏ <sos>
    if eos_idx in output_ids:
        output_ids = output_ids[:output_ids.index(eos_idx)]
    words = [fr_vocab.get_itos()[i] for i in output_ids]

```

```
return " ".join(words)
```

5. Đánh giá BLEU:

```
def compute_bleu(model, src_file, trg_file, device,
max_samples=None, use_beam=False, beam_width=3):
    smoothie = SmoothingFunction().method4
    scores = []
    with open(src_file, encoding='utf-8') as fsrc, open(trg_file,
encoding='utf-8') as ftrg:
        for i, (sline, tline) in enumerate(zip(fsrc, ftrg)):
            if max_samples and i >= max_samples:
                break
            sline = sline.strip()
            tline = tline.strip()
            if use_beam:
                pred = beam_translate(sline, model, device,
beam_width=beam_width)
            else:
                pred = translate(sline, model, device)
            reference = [tline.split()]
            hypothesis = pred.split()
            score = sentence_bleu(reference, hypothesis,
smoothing_function=smoothie)
            scores.append(score)
    return sum(scores)/len(scores) if scores else 0.0
```

6. Dịch thử:

```
if __name__ == "__main__":
    # Huấn luyện model
    model = run_training()

    # Load checkpoint
    ckpt_path = os.path.join(CHECKPOINT_DIR, 'best_model.pth')
    if os.path.exists(ckpt_path):
        data = torch.load(ckpt_path, map_location=DEVICE)
        enc = Encoder(len(en_vocab), EMBED_DIM, HIDDEN_DIM,
NUM_LAYERS, DROPOUT, PAD_IDX_EN)
        dec = Decoder(len(fr_vocab), EMBED_DIM, HIDDEN_DIM,
NUM_LAYERS, DROPOUT, PAD_IDX_FR)
        model = Seq2Seq(enc, dec, DEVICE).to(DEVICE)
        model.load_state_dict(data['model_state_dict'])
        print("Loaded best checkpoint.")

    # Dịch thử 5 câu
    TEST_FILE = './data/test_2016_flickr.en'
    with open(TEST_FILE, encoding='utf-8') as f:
        for i in range(5):
```

```

        s = f.readline().strip()
        print("EN:", s)
        print("FR_pred:", beam_translate(s, model, DEVICE,
beam_width=3))
        print("-"*30)

# Tính BLEU trên tập test
if os.path.exists(EN_TEST) and os.path.exists(FR_TEST):
    bleu = compute_bleu(model, EN_TEST, FR_TEST, DEVICE,
max_samples=200, use_beam=True, beam_width=3)
    print("BLEU on test:", bleu)

```


Tài liệu tham khảo

- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR.
- Luong, M. T., Pham, H., & Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP.
- Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
- PyTorch Team. *torch.nn.LSTM* — PyTorch Documentation.
<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
- PyTorch Tutorials. *Neural Machine Translation with Attention*.
https://pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html
- Elliott, D., et al. (2016). *Multi30K Dataset*.
<https://github.com/multi30k/dataset>